BÀI THỰC HÀNH 5

PricingService và Eureka Discovery

Minh chứng cài đặt và chạy thực tế trên laptop Windows

Thông tin	Nội dung
Sinh viên	Trương Đắc Kiên
MSSV / Lớp	2424802010029 / D24CNTT05
Ngày thực hiện	18/10/2026 (UTC+7)
Nền tảng	.NET 8.0.425, Docker Desktop Linux engine, PostgreSQL 16
Kết quả	PricingService UP trong Eureka; 12/12 test đạt; API tính phí = 98

Phạm vi hoàn thành

Task 1: tích hợp và đối chiếu ba gói PricingService.Api, PricingService.Test, PricingService.IntegrationTest của giảng viên với source bài 4. Đủ 17 file; 16 file giữ nguyên nội dung, một fixture được điều chỉnh cho môi trường kiểm thử.

Task 2: build và chạy Eureka Server; cấu hình Steeltoe Eureka Client; đăng ký PricingService; kiểm tra heartbeat, endpoint sức khỏe, API tính phí và phản hồi lỗi đầu vào.

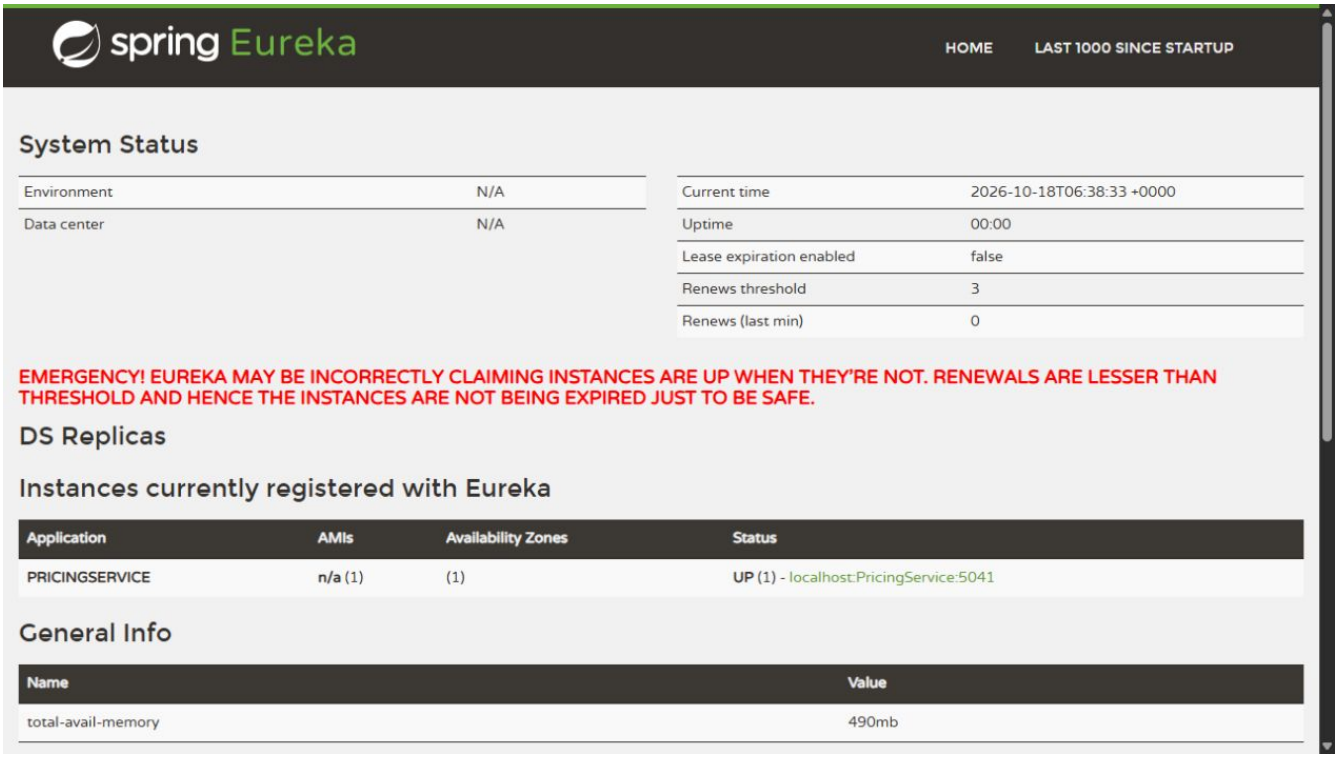
Mã nguồn nộp bài

github.com/KienTD204/Microservices-InsurancePortal-2424802010029/tree/student/lab5

Ảnh trong báo cáo được chụp từ ứng dụng đang chạy trên máy. Dữ liệu kiểm thử lấy từ TRX và phản hồi HTTP; các file gốc nằm trong submission/lab5/evidence.

01 / Eureka Server hoạt động

Địa chỉ: **http://localhost:8761**. Eureka chạy trong container eureka-server. PricingService chạy trên Windows và đăng ký instance **localhost:PricingService:5041**.



Hình 1. Giao diện Spring Eureka thật, có PRICINGSERVICE và trạng thái UP.

Xác minh ngoài giao diện

Kiểm tra	Kết quả
GET /eureka/apps/PRICINGSERVICE	Ứng dụng PRICINGSERVICE; status UP; port 5041
GET http://localhost:5041/health	HTTP 200, {"status":"UP"}
Heartbeat	PUT đến Eureka được trả HTTP 200; lease được gia hạn

Cảnh báo màu đỏ trong Eureka là chế độ tự bảo vệ khi số heartbeat thấp hơn ngưỡng của registry. Giữ nguyên cơ chế này trong lab. Việc service đang hoạt động được kiểm chứng thêm bằng heartbeat HTTP 200 và các lời gọi API thật, thay vì chỉ dựa vào nhãn UP.

Thời gian Eureka trong ảnh dùng UTC; thời gian kiểm tra lưu trong JSON dùng UTC+7.

02 / Cấu hình theo yêu cầu

Eureka Server - application.yml

```
server:
  port: 8761
eureka:
  server:
    waitTimeInMsWhenSyncEmpty: 0
  client:
    registerWithEureka: false
    fetchRegistry: false
```

PricingService - Startup.cs

```
using Steeltoe.Discovery.Client;

// ConfigureServices
services.AddDiscoveryClient(Configuration);

// Configure
app.UseDiscoveryClient();
```

PricingService - appsettings.json (các trường chính)

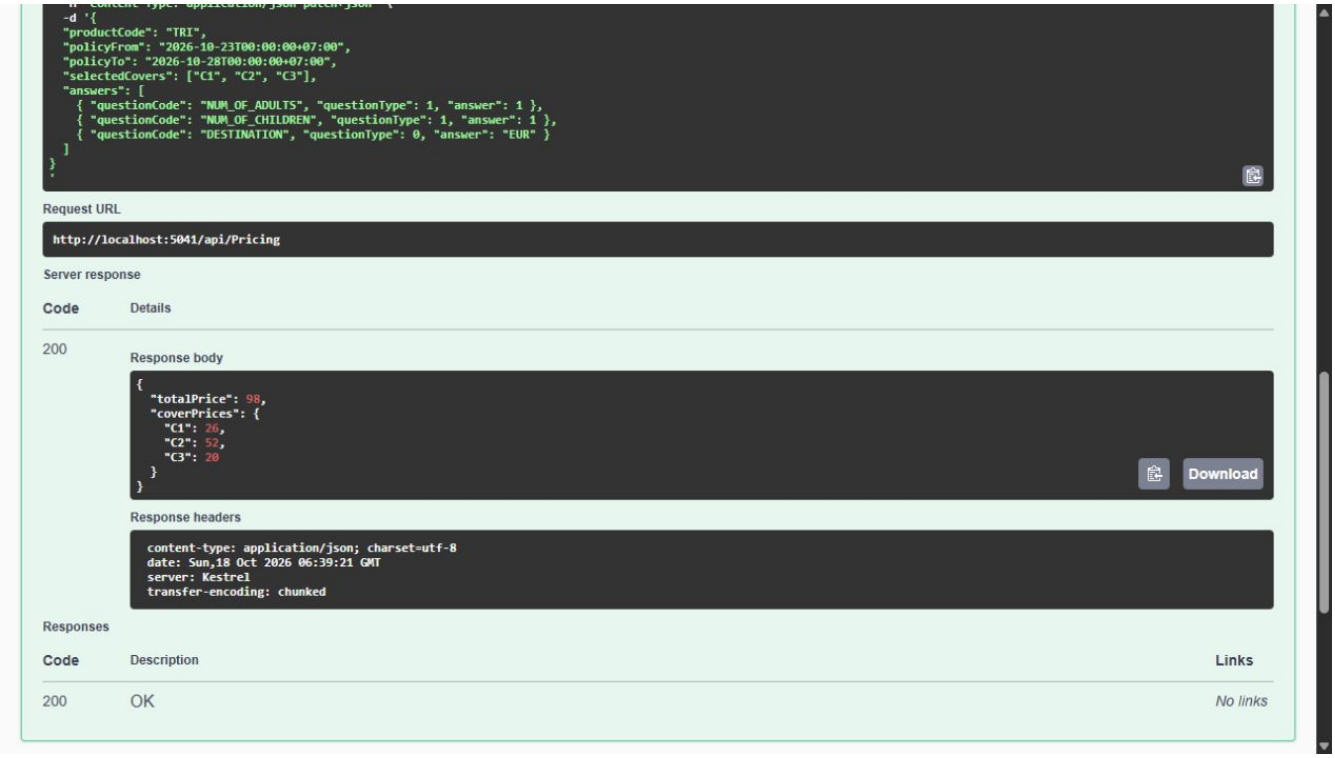
```
"Spring": { "Application": { "Name": "PricingService" } },
"Eureka": {
  "Client": {
    "ShouldRegisterWithEureka": true,
    "ShouldFetchRegistry": true,
    "ServiceUrl": "http://localhost:8761/eureka/",
    "ValidateCertificates": false
  },
  "Instance": {
    "AppName": "PricingService",
    "HostName": "localhost",
    "Port": 5041
  }
}
```

Điều chỉnh cổng: cổng 5040 trong đề bị Windows từ chối mở socket trên laptop này. Cổng 5041 được dùng thống nhất ở URL lắng nghe, launchSettings và instance Eureka.

Dùng Steeltoe.Discovery.ClientCore và Steeltoe.Discovery.Eureka 3.2.7. UseDiscoveryClient được giữ theo đề; với Steeltoe từ 3.1 trở lên, hosted service cũng tự khởi động discovery.

03 / API tính phí chạy thực tế

POST http://localhost:5041/api/Pricing
Sản phẩm TRI; 1 người lớn; 1 trẻ em; điểm đến EUR; chọn C1, C2, C3.



Hình 2. Swagger gửi yêu cầu đến PricingService và nhận HTTP 200, tổng phí 98.

Quyền lợi	Phép tính theo tariff	Phí
C1	1 người lớn x 26	26
C2	(1 người lớn + 1 trẻ em) x 26	52
C3	(1 người lớn + 1 trẻ em) x 10	20
Tổng	26 + 52 + 20	98

Kiểm tra dữ liệu không hợp lệ

Gửi JSON rỗng **{}** đến cùng endpoint nhận **HTTP 400**. Cả phản hồi hợp lệ và không hợp lệ được kiểm tra trong integration test của giảng viên và trong script verify-lab5.ps1.

Request mẫu đầy đủ: submission/lab5/request-travel.json.
Kết quả HTTP: submission/lab5/evidence/runtime-verification.json.

04 / Kiểm thử và tái lập kết quả

Nhóm kiểm tra	Đạt	Lỗi
PricingService.Test	10	0
PricingService.IntegrationTest	2	0
Tổng	12	0

Các test đã chạy

Test	Kết quả
PricingControllerTest.CommandIsProperlyValidated	Passed
PricingControllerTest.PriceForTravelPolicyIsCorrect	Passed
Commands.CalculatePriceHandlerShould.ReturnCorrectPrice	Passed
Commands.CalculatePriceHandlerShould.ThrowAnExceptionIfTheCommandsAnswersIsEmpty	Passed
Commands.CalculatePriceHandlerShould.ThrowAnExceptionIfTheCommandsProductCodeIsEmpty	Passed
Commands.CalculatePriceHandlerShould.ThrowAnExceptionIfTheCommandsSelectedCoversIsEmpty	Passed
Commands.CalculatePriceHandlerShould.ThrowExceptionsIfTheCommandIsDefault	Passed
Controllers.PricingControllerShould.CallMediatorWithCorrectParameters	Passed
Controllers.PricingControllerShould.ReturnJsonResult	Passed
Domain.TariffTest.CanCalculateCarPolicyPrice	Passed
Domain.TariffTest.CanCalculateHousePolicyPrice	Passed
Domain.TariffTest.CanCalculateTravelPolicyPrice	Passed

Build toàn bộ solution: **0 lỗi**. Lần full build có 55 cảnh báo; log incremental cuối có 17 cảnh báo dependency. Các cảnh báo NuGet/obsolete từ nền bài 4 vẫn được ghi nhận. Integration test dùng PostgreSQL 16 thật, không dùng mock database.

Tái lập tại thư mục gốc repository

```
./scripts/start-lab5.ps1
# Terminal thu hai:
./scripts/verify-lab5.ps1
```

Nộp eLearning: link GitHub nhánh student/lab5 cho Task 1 và file PDF này cho Task 2. Hạn trong đề: 18:00 ngày 18/10/2026.